

阿里妈妈高级前端工程师-AI产品方向

一面（技术基础） · 1-6

1. 简单介绍一下 Promise 的状态机机制，以及 all 和 allSettled 的区别。
2. 在AI 对话场景中，流式输出通常使用SSE 协议，它和 WebSocket 有什么区别？
3. 如何实现一个具有“打字机”效果的 AI 回复组件？
4. 说一下 TypeScript 中 interface 和 type的区别，以及在 AI 参数定义中如何选择？
5. 手写题：实现一个简单的防抖函数（debounce）。
6. 在 CSS 布局中，如何实现一个类似 ChatGPT 的左侧历史记录、右侧聊天区的左右布局？

二面（深入原理与项目） · 1-8

1. React 的 Fiber 架构解决了什么问题？在AI长文本渲染中有什么优势？
2. AI 产品的 Prompt 往往很长，前端如何优化 Prompt 的构建与传输效率？
3. 针对 AI生成的 Markdown内容，如何实现代码块的语法高亮和一键复制？
4. 你们是如何监控 AI产品的性能和质量的？有哪些关键指标？
5. 在 RAG（检索增强生成）流程中，前端可以参与哪些优化工作？
6. 实现一个大文件上传组件（用于 AI知识库），需要考虑哪些技术点？
7. 谈谈 Web Worker 在AI 前端应用中的使用场景。
8. 手写题：实现一个并发控制函数，要求同时最多执行 3个异步任务。

三面（架构与综合能力） 1-5

1. 如果让你设计一个通用的 AI Agent 搭建平台的前端架构，你会如何考虑？
2. 在 AI产品中，如何处理“幻觉”或错误信息对前端用户体验的影响？
3. 谈谈你对 AI时代前端工程师角色变化的看法。
4. 你们的项目中是如何处理多语言（i18n）的？在AI 翻译场景下有什么新思路？
5. 聊聊你在过去一年中遇到的最难的技术挑战，你是如何解决的？

一面（技术基础）

1. 简单介绍一下 Promise 的状态机机制，以及 all 和 allSettled 的区别。

难度：★

考点：异步编程基础

答案要点：

Promise 是异步编程的一种解决方案，它有三种状态：pending、fulfilled 和 rejected，状态切换不可逆。

- 状态转换：只能从 pending 变为 fulfilled 或从 pending 变为 rejected。
- Promise.all：接收一个 Promise 数组，只有全部成功才返回成功结果数组，只要有一个失败就立即返回失败。
- Promise.allSettled：无论每个 Promise 成功还是失败，都会等待所有 Promise 完成，并返回包含每个结果状态的对象数组。
- 适用场景：all 适用于强依赖关系的并行请求，allSettled 适用于互不影响的独立任务。

常见坑：

- 忽略 Promise.all 的短路特性，导致部分逻辑未执行。
- 在 allSettled 中忘记判断 status 字段直接取值。

追问：

- 如果想给 Promise.all 增加一个超时控制，你会怎么实现？

2. 在 AI 对话场景中，流式输出通常使用 SSE 协议，它和 WebSocket 有什么区别？

难度：★★

考点：网络协议与 AI 交互

答案要点：

SSE (Server-Sent Events) 是基于 HTTP 的单向通信协议，允许服务器向客户端推送数据，非常适合 AI 逐字生成的场景。

- 通信方向：SSE 是单向的（服务端到客户端），WebSocket 是全双工的双向通信。
- 协议基础：SSE 基于标准 HTTP 协议，无需握手升级，WebSocket 需要独立的协议转换。
- 断线重连：SSE 内置了自动重连机制，WebSocket 需要开发者手动实现。
- 数据格式：SSE 默认传输文本（UTF-8），WebSocket 支持二进制和文本。

常见坑：

- SSE 在 HTTP/1.1 下有浏览器连接数限制（通常为 6 个），需升级 HTTP/2。
- 忽略了 SSE 的数据缓冲区问题，导致前端无法实时接收到“流”。

追问：

- 为什么 AI 聊天机器人大多选择 SSE 而不是 WebSocket？

3. 如何实现一个具有“打字机”效果的 AI 回复组件？

难度：★★

考点：DOM 操作与性能

答案要点：

打字机效果的核心在于动态更新 DOM 内容，通常结合 SSE 返回的增量数据进行字符串拼接。

- 状态管理：使用 React 的 `useState` 存储已接收到的文本片段，并实时渲染。
- 性能优化：避免频繁操作 DOM，利用虚拟 DOM 的 diff 机制，或者在极高频更新时使用 `requestAnimationFrame`。
- 自动滚动：在内容增加时，判断用户是否处于底部，决定是否执行 `scrollIntoView`。
- 格式处理：实时将 Markdown 字符串解析为 HTML，需注意解析器的增量渲染能力。

常见坑：

- 频繁渲染导致页面卡顿，尤其是在长对话场景下。
- 自动滚动逻辑冲突，用户手动向上翻阅时被强制拉回底部。

追问：

- 如果后端返回速度极快，如何平滑地展示打字效果？

4. 说一下 TypeScript 中 `interface` 和 `type` 的区别，以及在 AI 参数定义中如何选择？

难度：★

考点：TS 基础

答案要点：

`interface` 主要用于定义对象的结构，支持声明合并；`type` 则是类型别名，支持联合类型、元组等更复杂的定义。

- 扩展方式：`interface` 通过 `extends` 继承，`type` 通过 `&` 交叉类型实现。
- 声明合并：同名 `interface` 会自动合并，`type` 不允许同名定义。
- 适用场景：定义 AI 插件的配置接口建议用 `interface`，定义模型名称（如 `'gpt-3.5' | 'gpt-4'`）建议用 `type`。
- 性能：`interface` 的类型检查性能通常略优于 `type`。

常见坑：

- 在需要联合类型的地方强行使用 `interface` 导致逻辑复杂。
- 忽略了 `interface` 自动合并可能带来的全局污染。

追问：

- 如何利用 TS 的泛型来封装一个通用的 AI API 响应结构？

5. 手写题：实现一个简单的防抖函数（debounce）。

难度：★

考点：基础手写

答案要点：

防抖是指在事件被触发 n 秒后再执行回调，如果在这 n 秒内又被触发，则重新计时。

- 核心原理：利用闭包保存 timer 变量。
- 清除定时器：每次触发时先 clearTimeout。
- 立即执行：可选参数支持第一次触发时是否立即执行。
- 返回值：确保 this 指向和参数正确传递。

代码块

```
1 function debounce(fn, delay) {
2   let timer = null;
3   return function(...args) {
4     if (timer) clearTimeout(timer);
5     timer = setTimeout(() => {
6       fn.apply(this, args);
7     }, delay);
8   };
9 }
```

常见坑：

- 忘记处理 this 指向。
- 忘记处理 arguments 参数。

6. 在 CSS 布局中，如何实现一个类似 ChatGPT 的左侧历史记录、右侧聊天区的左右布局？

难度：★

考点：CSS 布局

答案要点：

通常使用 Flexbox 或 Grid 布局来实现响应式的侧边栏结构。

- Flex 布局：父容器 display: flex，左侧固定宽度，右侧 flex: 1。
- 溢出处理：右侧聊天区设置 overflow-y: auto，并确保高度撑满容器。
- 响应式：利用媒体查询在移动端隐藏侧边栏，改为抽屉式菜单。
- 现代方案：使用 CSS Grid 的 grid-template-columns: 260px 1fr。

常见坑：

- 内部长文本导致 Flex 容器被撑开，需设置 min-width: 0。
- 移动端 100vh 在某些浏览器（如 Safari）包含工具栏，导致布局错位。

追问：

- 如何实现侧边栏的拖拽调宽功能？

二面（深入原理与项目）

1. React 的 Fiber 架构解决了什么问题？在 AI 长文本渲染中有什么优势？

难度：★★★★

考点：框架原理

答案要点：

Fiber 架构通过将渲染工作拆分为小片并支持优先级调度，解决了 React 在处理大量 DOM 更新时的掉帧问题。

- 核心机制：引入了可中断的异步渲染，将同步更新变为增量更新。
- 调度器（Scheduler）：根据任务优先级（如用户输入高于数据流更新）决定执行顺序。
- AI 场景优势：AI 流式返回时，频繁的 state 更新不会阻塞 UI 响应，保证了用户输入的流畅性。
- 双缓存技术：在内存中构建 Fiber 树，减少了直接操作真实 DOM 的开销。

常见坑：

- 误以为 Fiber 提高了渲染绝对速度，实际上它提升的是感知流畅度。
- 在 render 阶段执行了有副作用的操作。

追问：

- 如何在 React 中手动控制某个更新任务的优先级？

2. AI 产品的 Prompt 往往很长，前端如何优化 Prompt 的构建与传输效率？

难度：★★

考点：AI 业务优化

答案要点：

前端在构建 Prompt 时应遵循模块化和精简原则，减少 Token 消耗并提升响应速度。

- 模板化管理：将 System Prompt、User Context、Few-shot 示例拆分，按需组合。
- 缓存策略：对静态的上下文或历史信息进行本地缓存，避免重复传输。
- Token 预估：在发送前使用 tiktoken 等库进行前端估算，超出限制时进行截断或压缩。
- 变量注入：设计严谨的占位符替换逻辑，防止用户输入破坏 Prompt 结构。

常见坑：

- 直接拼接字符串导致 Prompt 注入攻击。
- 忽略了 Token 限制，导致请求被模型后端拒绝。

追问：

- 什么是 Prompt 注入？前端能做哪些预防？

3. 针对 AI 生成的 Markdown 内容，如何实现代码块的语法高亮和一键复制？

难度：★★

考点：组件设计与生态

答案要点：

需要结合 Markdown 解析插件（如 react-markdown）和高亮库（如 prismjs 或 highlight.js）进行定制化开发。

- 自定义渲染器：重写 Markdown 组件中的 code 节点渲染逻辑。
- 语法高亮：根据语言类型动态加载高亮样式，或使用 Web Worker 处理大型代码块解析。
- 交互增强：在代码块右上角注入 CopyButton 组件，利用 clipboard API 实现复制。
- 性能：对于超长代码块，考虑使用虚拟列表或延迟渲染。

常见坑：

- 样式冲突导致高亮失效。
- 复制功能在非 HTTPS 环境下失效。

追问：

- 如果 AI 正在流式输出代码，你的高亮逻辑会频繁闪烁吗？怎么解决？

4. 你们是如何监控 AI 产品的性能和质量的？有哪些关键指标？

难度：★★★

考点：工程化与监控

答案要点：

AI 产品的监控不仅包含传统前端指标，还需关注模型交互特有的性能数据。

- 性能指标：TTFT（首 Token 响应时间）、TPS（每秒生成 Token 数）、请求成功率。
- 质量指标：用户点赞/点踩率、会话中断率、Prompt 命中率。
- 异常监控：流式连接中断、模型超时、敏感词拦截触发。
- 埋点方案：记录完整的 Trace ID，方便前后端链路追踪定位问题。

常见坑：

- 只监控 HTTP 状态码，忽略了业务层面的“回答错误”。
- 监控数据量过大导致前端性能下降。

追问：

- 如何在前端捕获并上报流式输出过程中的断网异常？

5. 在 RAG（检索增强生成）流程中，前端可以参与哪些优化工作？

难度：★★★

考点：AI 架构理解

答案要点：

前端在 RAG 中主要负责知识库管理、检索结果展示以及引用溯源的交互。

- 引用溯源：将模型生成的引用标记（如 [1]）与原始文档片段关联，点击可跳转查看原文。
- 检索干预：提供搜索关键词微调界面，允许用户在生成前确认检索到的上下文。
- 文档解析预览：在上传文档时，前端进行初步的格式校验和分段预览。
- 交互反馈：展示检索过程的状态（如“正在搜索知识库...”），缓解用户焦虑。

常见坑：

- 引用链接失效或指向错误的文档片段。
- 检索到的上下文过长，前端展示不全。

追问：

- 如何在 UI 上优雅地展示模型对多个文档的综合引用？

6. 实现一个大文件上传组件（用于 AI 知识库），需要考虑哪些技术点？

难度：★★

考点：文件处理

答案要点：

大文件上传需要解决稳定性、速度和服务器压力问题。

- 分片上传：利用 Blob.slice 将文件切块，并发上传。
- 断点续传：通过文件 MD5 值（计算 Hash）在服务端查询已上传的分片。
- 秒传：如果服务端已存在该 Hash，直接返回成功。
- 进度反馈：监听 xhr.upload.onprogress 或使用 axios 的上传进度回调。
- 性能优化：使用 Web Worker 计算大文件的 MD5，避免阻塞主线程。

常见坑：

- 计算 Hash 导致浏览器卡死。
- 并发请求过多导致浏览器挂起。

追问：

- 如果上传过程中网络切换了，你的组件如何自动恢复？

7. 谈谈 Web Worker 在 AI 前端应用中的使用场景。

难度：★★

考点：浏览器性能

答案要点：

Web Worker 允许在后台线程执行耗时任务，避免阻塞 UI 渲染。

- 场景一：离线模型运行（如使用 Transformers.js 在浏览器端运行小模型）。
- 场景二：大规模数据的预处理、向量计算或复杂的 Markdown 解析。
- 场景三：大文件的 Hash 计算（MD5/SHA256）。

- 通信机制：通过 `postMessage` 传递数据，注意 Transferable Objects 以提升传输效率。

常见坑：

- 在 Worker 中尝试操作 DOM。
- 频繁的通信开销超过了计算节省的时间。

追问：

- SharedWorker 和普通 Worker 有什么区别？

8. 手写题：实现一个并发控制函数，要求同时最多执行 3 个异步任务。

难度：★★

考点：异步编程进阶

答案要点：

核心思路是维护一个正在执行的任务队列，当一个任务完成后，自动触发下一个任务。

- 计数器：记录当前正在运行的任务数。
- 任务队列：存储待执行的任务。
- 递归执行：任务完成后，计数器减一，并从队列中取新任务执行。

代码块

```
1  async function limitConcurrency(tasks, limit) {
2    const results = [];
3    const executing = new Set();
4    for (const task of tasks) {
5      const p = Promise.resolve().then(() => task());
6      results.push(p);
7      executing.add(p);
8      const clean = () => executing.delete(p);
9      p.then(clean).catch(clean);
10     if (executing.size >= limit) {
11       await Promise.race(executing);
12     }
13   }
14   return Promise.all(results);
15 }
```

常见坑：

- 无法正确处理任务失败的情况。
- 结果顺序与输入顺序不一致。

三面（架构与综合能力）

1. 如果让你设计一个通用的 AI Agent 搭建平台的前端架构，你会如何考虑？

难度：★★★

考点：系统架构设计

答案要点：

设计 Agent 平台需要平衡灵活性、可扩展性和用户体验，通常采用低代码或工作流引擎的思想。

- 核心引擎：基于 JSON Schema 或图结构（DAG）定义 Agent 的编排逻辑。
- 组件化设计：将 LLM 调用、工具调用（Tools）、记忆模块（Memory）抽象为标准节点。
- 状态管理：使用全局状态管理（如 Zustand 或 Redux）处理复杂的节点间数据流。
- 可视化画布：集成 React Flow 或 X6 实现拖拽式编排，支持实时调试。
- 扩展机制：提供标准 SDK，允许开发者自定义插件节点。

常见坑：

- 节点间循环依赖导致死循环。
- 画布渲染大量节点时的性能问题。

追问：

- 如何实现 Agent 执行过程中的实时单步调试预览？

2. 在 AI 产品中，如何处理“幻觉”或错误信息对前端用户体验的影响？

难度：★★

考点：产品思维与交互

答案要点：

前端可以通过交互设计引导用户理性看待 AI 输出，并提供纠错机制。

- 显著提示：在输入框或回复区标注“AI 生成内容仅供参考”。
- 反馈闭环：提供点赞、点踩、重新生成、手动编辑功能。
- 引用透明：强制要求 AI 标注信息来源，前端提供便捷的核对入口。
- 渐进式展示：对于高风险操作，增加人工确认环节。

常见坑：

- 过度信任 AI 输出，缺乏兜底逻辑。
- 反馈机制太隐蔽，用户无法有效纠错。

追问：

- 如果模型返回了违反安全策略的内容，前端应该如何拦截并展示？

3. 谈谈你对 AI 时代前端工程师角色变化的看法。

难度：★★

考点：技术视野

答案要点：

前端工程师正从单纯的 UI 开发转向“全栈交互”和“模型应用”工程师。

- 技能边界：需要掌握 Prompt Engineering、了解模型参数、甚至简单的 Python/LangChain。
- 交互变革：从传统的点击交互转向自然语言交互（LUI），关注对话流的体验。
- 提效工具：利用 Copilot 等工具提升编码效率，将精力转向更复杂的架构设计。
- 价值体现：前端更贴近用户，是 AI 能力落地的最后一步，决定了产品的最终体验。

常见坑：

- 盲目追求 AI 技术而忽略了前端基础建设。
- 认为 AI 会取代前端，产生职业焦虑。

追问：

- 你最近在关注哪些 AI 相关的技术趋势？

4. 你们的项目中是如何处理多语言（i18n）的？在 AI 翻译场景下有什么新思路？

难度：★★

考点：国际化与 AI 应用

答案要点：

传统方案依赖静态语言包，AI 时代可以实现更智能的动态翻译和上下文感知。

- 基础方案：使用 i18next 等库，配合自动化脚本提取词条。
- AI 赋能：利用大模型进行语境感知的翻译，解决传统翻译生硬的问题。
- 动态翻译：对于用户生成的动态内容，接入翻译 API 实现实时多语言切换。
- 质量控制：AI 预翻译 + 人工审核（HITL）的混合模式。

常见坑：

- 忽略了不同语言的文本长度对布局的影响。
- 动态翻译的延迟导致用户体验中断。

追问：

- 如何保证 AI 翻译的术语一致性？

5. 聊聊你在过去一年中遇到的最难的技术挑战，你是如何解决的？

难度：★★★

考点：综合解决问题能力

答案要点：

此题没有标准答案，重点考察逻辑表达和抗压能力。

- 背景：清晰描述业务场景和技术痛点（如：AI 响应慢、长列表卡顿、架构耦合）。

- 方案对比：列举尝试过的多种方案，并说明最终选型的理由。
- 实施过程：遇到的具体阻碍及如何突破（如：查源码、社区求助、实验论证）。
- 最终结果：用数据说话（如：性能提升 50%、用户投诉率下降）。
- 总结反思：沉淀了什么方法论或通用组件。

常见坑：

- 描述过于笼统，没有技术细节。
- 方案选择理由不充分，显得随大流。

追问：

- 如果现在让你重新做一遍，你会怎么优化？